



Data Structure

(Course Code: 25B11CI318)

K-ary Trees

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

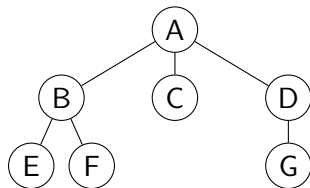
What is a K-ary Tree?

Definition

A **K-ary tree** is a rooted tree in which every node has **at most K children**.

$$0 \leq \text{number of children of a node} \leq K$$

- $K = 2$: Binary tree
- $K = 3$: Ternary tree
- $K = 4$: Quaternary tree
- General case: tree



Basic Terminology

Root: topmost node of the tree.

Parent: node having one or more children.

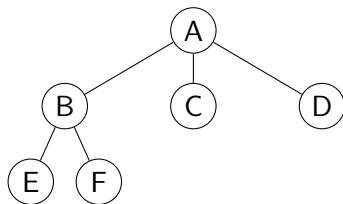
Child: node directly below another node.

Siblings: nodes sharing the same parent.

Leaf: node having no children.

Internal node: node having at least one child.

Degree: number of children of a node.

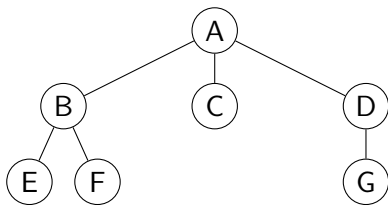


A is root; *B*, *C*, *D* are siblings; *C*, *D*, *E*, *F* are leaves.

Level, Depth, and Height

Level / Depth

If the root is at level 0, then the depth of a node is the number of edges from the root to that node.



- Level 0: *A*
- Level 1: *B, C, D*
- Level 2: *E, F, G*

Height

The height h is the number of edges on the longest root-to-leaf path.

For this tree, $h = 2$.

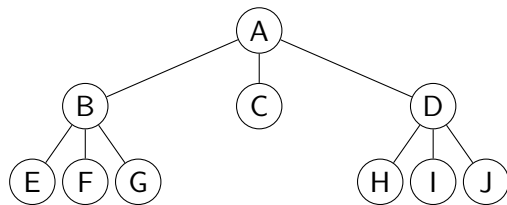
Major Types of K-ary Trees

Type	Main Property
General	Every node has at most K children.
Full	Every internal node has exactly K children; each node has either 0 or K children.
Complete	Every level is full except possibly the last; the last level is filled from left to right.
Perfect	Every internal node has exactly K children and all leaves are at the same level.

Important

Every perfect tree is full and complete, but a full tree need not be perfect.

Example: Full 3-ary Tree



- $K = 3$
- Internal nodes: A, B, D
- Every internal node has exactly 3 children.
- Leaves have 0 children.

$$\text{children} \in \{0, K\}$$

Maximum Nodes at a Level

For a tree whose root is at level 0, the maximum number of nodes at level i is

$$N_i = K^i$$

Level i	Maximum nodes	For $K = 3$
0	K^0	1
1	K^1	3
2	K^2	9
3	K^3	27
4	K^4	81

Reason

Each node can generate at most K children at the next level.

Maximum Number of Nodes for Height h

For maximum occupancy, all levels from 0 to h are completely filled:

$$N_{\max} = 1 + K + K^2 + \dots + K^h$$

Using the sum of a geometric progression,

$$N_{\max} = \frac{K^{h+1} - 1}{K - 1}, \quad K > 1$$

Example

For a 3-ary tree of height 3:

$$N_{\max} = \frac{3^4 - 1}{3 - 1} = \frac{81 - 1}{2} = 40$$

Minimum Height for N Nodes

Starting from

$$N \leq \frac{K^{h+1} - 1}{K - 1},$$

we obtain

$$K^{h+1} \geq N(K - 1) + 1.$$

Hence,

$$h_{\min} = \lceil \log_K (N(K - 1) + 1) \rceil - 1$$

Interpretation

A larger branching factor K generally reduces the required height for the same number of nodes.

Full K-ary Tree: Leaves and Internal Nodes

Let

I = number of internal nodes, L = number of leaves.

Each internal node contributes exactly K child edges, so the total number of edges is KI .
But every tree with N nodes has $N - 1$ edges, and

$$N = I + L.$$

Therefore,

$$KI = I + L - 1$$

and hence

$$L = (K - 1)I + 1$$

Full K-ary Tree: Other Useful Relations

From

$$L = (K - 1)I + 1,$$

we obtain

$$N = I + L = KI + 1.$$

Thus,

$$N = KI + 1$$

and

$$I = \frac{N - 1}{K}$$

The number of leaves in terms of total nodes is

$$L = \frac{(K - 1)N + 1}{K}.$$

Note

These relations apply specifically to a **full** tree.

Perfect K-ary Tree

In a perfect tree of height h :

$$\text{Leaves: } L = K^h,$$

$$\text{Internal nodes: } I = \frac{K^h - 1}{K - 1},$$

$$\text{Total nodes: } N = \frac{K^{h+1} - 1}{K - 1}.$$

Example: $K = 3, h = 4$

$$L = 3^4 = 81, \quad I = \frac{81 - 1}{2} = 40, \quad N = 121.$$

Number of Edges

A fundamental property of every tree is

$$E = N - 1$$

where

- E = number of edges,
- N = number of nodes.

Example

If a tree contains 50 nodes,

$$E = 50 - 1 = 49.$$

This relation is independent of the value of K .

Array Representation

A complete tree can be stored efficiently in an array.

For **0-based indexing**, if a node is stored at index i :

$$j\text{th child} = Ki + j, \quad 1 \leq j \leq K$$

$$\text{parent}(i) = \left\lfloor \frac{i-1}{K} \right\rfloor, \quad i > 0$$

Example: $K = 3$, node index $i = 2$

Children are located at

$$3(2) + 1 = 7, \quad 3(2) + 2 = 8, \quad 3(2) + 3 = 9.$$

The parent of node 8 is

$$\left\lfloor \frac{8-1}{3} \right\rfloor = 2.$$

Linked Representation

A node may store a fixed-size array of child pointers:

Conceptual Structure

```
Node {  
    data  
    child[0], child[1], ..., child[K-1]  
}
```

Fixed array of pointers

- direct access to child positions;
- simple implementation;
- may waste space in sparse trees.

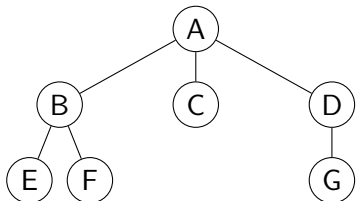
Dynamic child list

- stores only existing children;
- more memory-efficient for sparse trees;
- maintain $|\text{children}| \leq K$.

Preorder Traversal

Rule

Visit the node first, then recursively visit all children from left to right.



A, B, E, F, C, D, G

Pseudocode

```
visit(node)
for each child:
    Preorder(child)
```

Postorder and Level-Order Traversal

Postorder

- Traverse all children first.
- Visit the current node last.

For the same example tree:

`E, F, B, C, G, D, A`

Level-order (BFS)

- Uses a queue.
- Visits nodes level by level.

Traversal:

`A, B, C, D, E, F, G`

Inorder?

There is no single universally accepted inorder traversal for a general tree.

Complexity Analysis

Let N be the number of nodes and h the height.

Operation / Traversal	Time	Auxiliary Space
Preorder DFS	$O(N)$	$O(h)$ recursion
Postorder DFS	$O(N)$	$O(h)$ recursion
Level-order BFS	$O(N)$	up to width of tree
Search in unordered tree	$O(N)$ worst case	depends on method

For a balanced tree,

$$h = O(\log_K N).$$

A larger K generally makes the tree shallower.

Binary Tree vs K-ary Tree

Property	Binary Tree	Tree
Maximum children	2	K
Max. nodes at level i	2^i	K^i
Max. nodes of height h	$2^{h+1} - 1$	$\frac{K^{h+1} - 1}{K - 1}$
Balanced height	$O(\log_2 N)$	$O(\log_K N)$
Inorder traversal	well-defined	not unique in general

Advantages and Limitations

Advantages

- smaller height than binary trees for large K ;
- natural model for multi-way relationships;
- useful for hierarchical data;
- fewer levels may reduce traversal depth;
- effective for storage systems with large block sizes.

Limitations

- more child references may be required;
- fixed child arrays may waste memory;
- larger K means more children to inspect;
- some binary-tree algorithms do not generalize directly.

Applications of K-ary Trees

- **File systems:** directories containing several subdirectories/files.
- **Artificial intelligence:** search spaces with branching factor K .
- **Game trees:** a state can have multiple legal moves.
- **Database indexing:** multi-way search structures such as B-trees and B+ trees.
- **Priority queues:** d-ary heaps generalize binary heaps.
- **Network routing:** hierarchical routing and multicast structures.
- **Parallel computing:** one task distributes work to several child tasks.

Solved Example 1

Question: A perfect 3-ary tree has height 4. Find the number of leaves, internal nodes, and total nodes.

Given $K = 3$ and $h = 4$:

$$L = K^h = 3^4 = 81.$$

$$I = \frac{K^h - 1}{K - 1} = \frac{81 - 1}{2} = 40.$$

$$N = I + L = 40 + 81 = 121.$$

Answer

$$L = 81, \quad I = 40, \quad N = 121$$

Solved Example 2

Question: A full 5-ary tree contains 21 internal nodes. Find the number of leaves and total nodes.

Given $K = 5$ and $I = 21$:

$$L = (K - 1)I + 1 = (5 - 1)(21) + 1 = 85.$$

$$N = KI + 1 = 5(21) + 1 = 106.$$

Answer

$$L = 85, \quad N = 106$$

Solved Example 3

Question: What is the maximum number of nodes in a 4-ary tree of height 5?

$$K = 4, \quad h = 5$$

$$\begin{aligned} N_{\max} &= \frac{K^{h+1} - 1}{K - 1} = \frac{4^6 - 1}{4 - 1} \\ &= \frac{4096 - 1}{3} = \frac{4095}{3} = 1365. \end{aligned}$$

Answer

$$N_{\max} = 1365$$

Thank You